

LABORATORIO 09

La libreta desde Scala

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

Este laboratorio responde una pregunta concreta. ¿Se puede usar Iceberg desde Scala, sin Spark? Vas a correr, desde una celda de tu cuaderno, un programa Scala que abre tu tabla, la lee, le escribe una fila y te dice dónde viven sus archivos. En ese programa no hay Spark en ninguna parte.

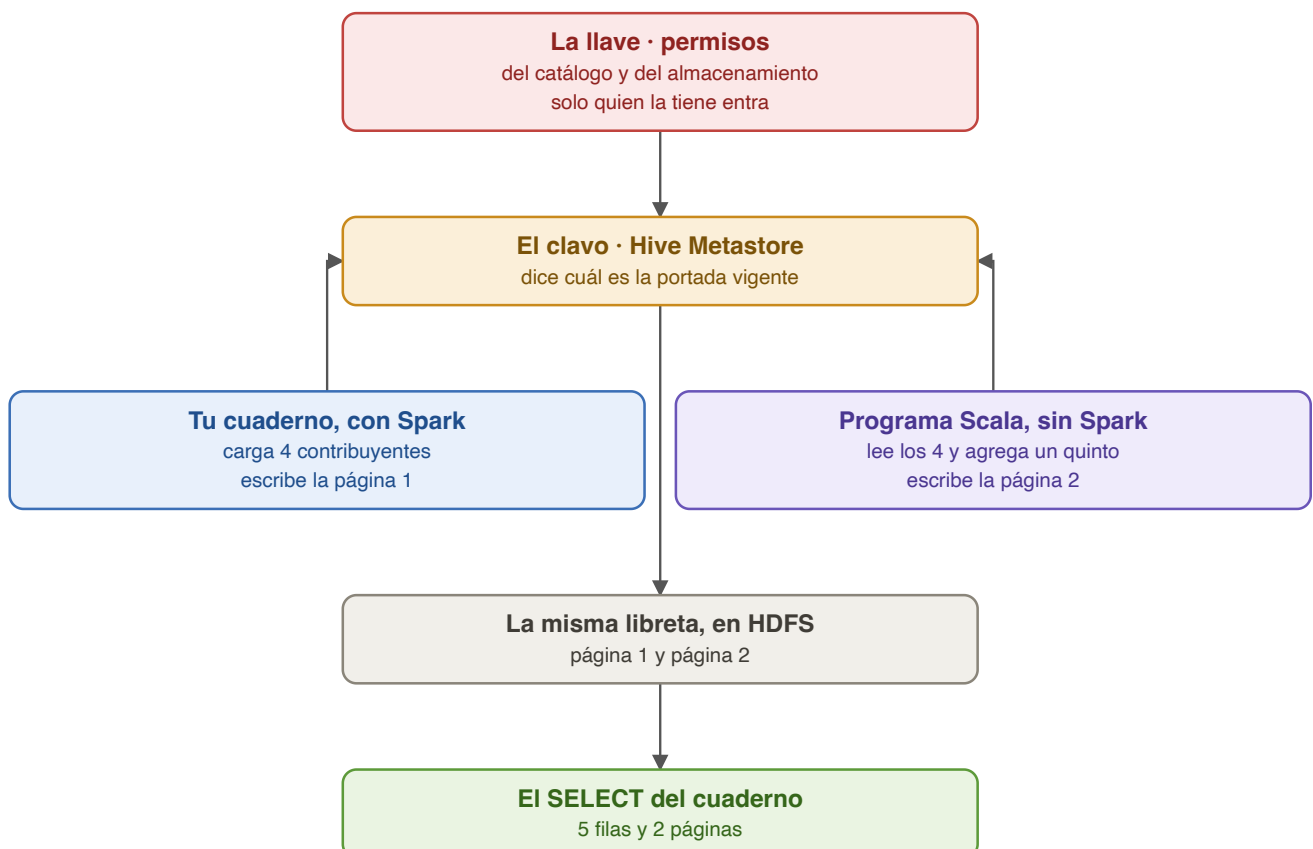
2 · El problema, en el almacén

El almacén tiene un contador, y el contador trabaja con su propio sistema. Necesita leer la libreta y anotar un pago. En casi todos los negocios la libreta está guardada en el cajón del almacenero, así que el contador tiene que pedirle a él cada dato y cada anotación. Todo pasa por las manos del almacenero, y el almacenero se transforma en un cuello de botella. Si mañana cambian al almacenero, hay que volver a acordar todo desde cero.

3 · El problema en términos técnicos

En una base de datos tradicional, los datos quedan encerrados en el motor. Cada motor guarda en su propio formato y solo ese motor los puede leer. Si otro programa necesita esos datos, se tiene que conectar al motor o alguien se los exporta. De ahí salen las copias, los procesos de extracción y los números que no cuadran entre un sistema y otro. Cambiar de motor significa migrar todos los datos.

4 · El diagrama



La llave · permisos (rojo). Son los permisos. Iceberg no decide quién entra. Eso lo deciden los permisos del catálogo y del almacenamiento. El que no tiene llave no ve la libreta, ni desde Spark ni desde Scala.

El clavo · Hive Metastore (amarillo). Es el catálogo. De cada tabla guarda una sola cosa, dónde está su portada vigente. Cualquiera que quiera leer o escribir pasa primero por aquí a preguntar.

Tu cuaderno, con Spark (azul). Crea la tabla y carga cuatro contribuyentes. Eso deja la página 1.

Programa Scala, sin Spark (morado). Es un programa empaquetado en un solo archivo, que no trae Spark. Va al mismo clavo, lee los cuatro contribuyentes y agrega un quinto, Litre Transportes SpA. Eso deja la página 2.

La misma libreta, en HDFS (gris). La libreta es una sola. La página 1 la escribió Spark y la página 2 la escribió Scala. Para la libreta son iguales.

El SELECT del cuaderno (verde). Es la prueba. Vuelves al cuaderno, consultas y aparecen cinco filas y dos páginas. Spark lee a Litre Transportes sin saber que la escribió otro programa.

5 · La solución

En el almacén, el contador tiene llave de la bodega. No la tiene cualquiera, se la dio el dueño. Con esa llave entra, mira el clavo, descuelga la libreta, lee y anota su hoja con las mismas reglas que todos, sin depender del almacenero.

En Iceberg pasa lo mismo. La tabla no está encerrada en un motor. Es un formato de archivos público más una biblioteca que sabe leerlo, y **no existe un servidor de Iceberg** al que haya que pedirle las cosas. La llave no la entrega Iceberg. La entregan los permisos del catálogo, que es el Hive Metastore, y los del almacenamiento, que es HDFS. Un programa con esos permisos y con la biblioteca lee y escribe la tabla directamente. Cuando escribe, sigue las mismas reglas que Spark. Su anotación queda como una página nueva y completa, o no queda.

6 · Los pasos

- **Paso 0.** Creas la tabla desde el cuaderno y cargas cuatro contribuyentes. Queda una página.
- **Paso 1.** Lees el programa Scala, sin correrlo.
- **Paso 2.** Corres el programa. Lee los cuatro contribuyentes y agrega a Litre Transportes.
- **Paso 3.** Vuelves al cuaderno y compruebas que hay cinco filas y dos páginas.
- **Paso 4.** Ves qué hace falta declarar para hacer lo mismo en un proyecto propio, sin Spark.
- **Paso 5.** Ves cómo se conecta un Spark cualquiera al mismo catálogo.

Si ya corriste este laboratorio y quieres partir limpio, borra su tabla con el comando de abajo, desde la carpeta del repositorio. La celda 0.2 también la borra, así que no es obligatorio.

```
bash
bin/reiniciar-lab.sh 09
```

Paso 0 · La tabla de hoy

Celda 0.1 · Ponerte en tu espacio

La pregunta. ¿En qué espacio vas a trabajar?

Por qué ahora. Antes de crear nada hay que decirle a Spark dónde quieres que quede todo. Si no se lo dices, la tabla puede terminar en otro espacio.

En el almacén. Es elegir tu estante en la bodega. Todo lo que crees desde aquí queda en ese estante.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Lo que va después de dos guiones lo ignora el motor, y sirve para saber qué celda es.
- `USE` cambia el espacio con el que trabajas.
- `mi_espacio` es tu espacio. Desde aquí, cuando nombres una tabla sin decir su espacio, Spark la busca en `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:08:42 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La última línea dice que la sentencia se ejecutó. Las tres de arriba son avisos de Spark al arrancar, y salen solo en la primera celda.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:08:42` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.
- `Listo. La sentencia se ejecutó.` es el aviso que pone el cuaderno cuando una sentencia no devuelve filas.

Celda 0.2 · Partir de cero

La pregunta. ¿Y si la tabla ya existe?

Por qué ahora. Al final vas a contar dos páginas exactas. Si quedó una tabla de una corrida anterior, con páginas viejas, la cuenta no calza.

En el almacén. Es descolgar del clavo cualquier libreta vieja con este nombre, para colgar una nueva en blanco. Si no hay ninguna, no pasa nada.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla del catálogo.
- `IF EXISTS` la borra solo si existe. Si no existe, no reclama.
- `contribuyentes_lab09` es el nombre de la tabla. Va sin el espacio adelante porque ya estás en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS contribuyentes_lab09
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. El aviso es el mismo exista o no la tabla.

Celda 0.3 · Crear la tabla

La pregunta. ¿Dónde va a escribir el programa Scala?

Por qué ahora. Hace falta una tabla que sea de los dos, del cuaderno y del programa. La creas desde Spark y en formato Iceberg. Eso último es lo que importa, porque el programa Scala no sabe nada de Spark pero sí sabe leer Iceberg.

En el almacén. Es colgar en el clavo una libreta nueva en blanco, con las columnas ya rayadas para el RUT, la razón social y el segmento.

La sentencia, parte por parte.

- `CREATE TABLE contribuyentes_lab09` crea la tabla con ese nombre, en `mi_espacio`.
- `rut STRING` es el RUT, guardado como texto porque lleva guion y dígito verificador.
- `razon_social STRING` es el nombre de la empresa.
- `segmento STRING` es el tamaño del contribuyente, MICRO, PEQUENA, MEDIANA o GRANDE.
- `USING iceberg` deja la tabla en formato Iceberg, que es el que el programa Scala sabe abrir. Registra la tabla en el catálogo y escribe su primera portada, todavía sin datos ni páginas.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE contribuyentes_lab09 (
  rut          STRING,
  razon_social STRING,
  segmento     STRING
) USING iceberg
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla existe y está vacía.

Celda 0.4 · Cargar cuatro contribuyentes

La pregunta. ¿Qué va a encontrar el programa cuando abra la tabla?

Por qué ahora. El programa tiene que tener algo que leer. Además esta carga deja una sola página, y al final vas a contar dos. La segunda no la escribe Spark.

En el almacén. Es escribir la primera página de la libreta, cuatro contribuyentes en un solo papelito, y anotar en la portada que esta página es la vigente.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab09` agrega filas a la tabla.
- `VALUES` trae las filas escritas a mano, cada una entre paréntesis, con el RUT, la razón social y el segmento en el orden de las columnas.
- Son cuatro filas en un solo `INSERT`, y por eso queda una sola página.

```
sql
%%sql
-- Celda 0.4
INSERT INTO contribuyentes_lab09 VALUES
  ('77746521-K', 'Pehuen Logistica EIRL', 'MICRO'),
  ('77884562-8', 'Araucaria Ferretería SpA', 'PEQUENA'),
  ('78800840-6', 'Copihue Maquinarias EIRL', 'MEDIANA'),
  ('76171162-8', 'Huemul Alimentos EIRL', 'GRANDE')
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Las cuatro filas entraron. La libreta tiene ahora su página 1.

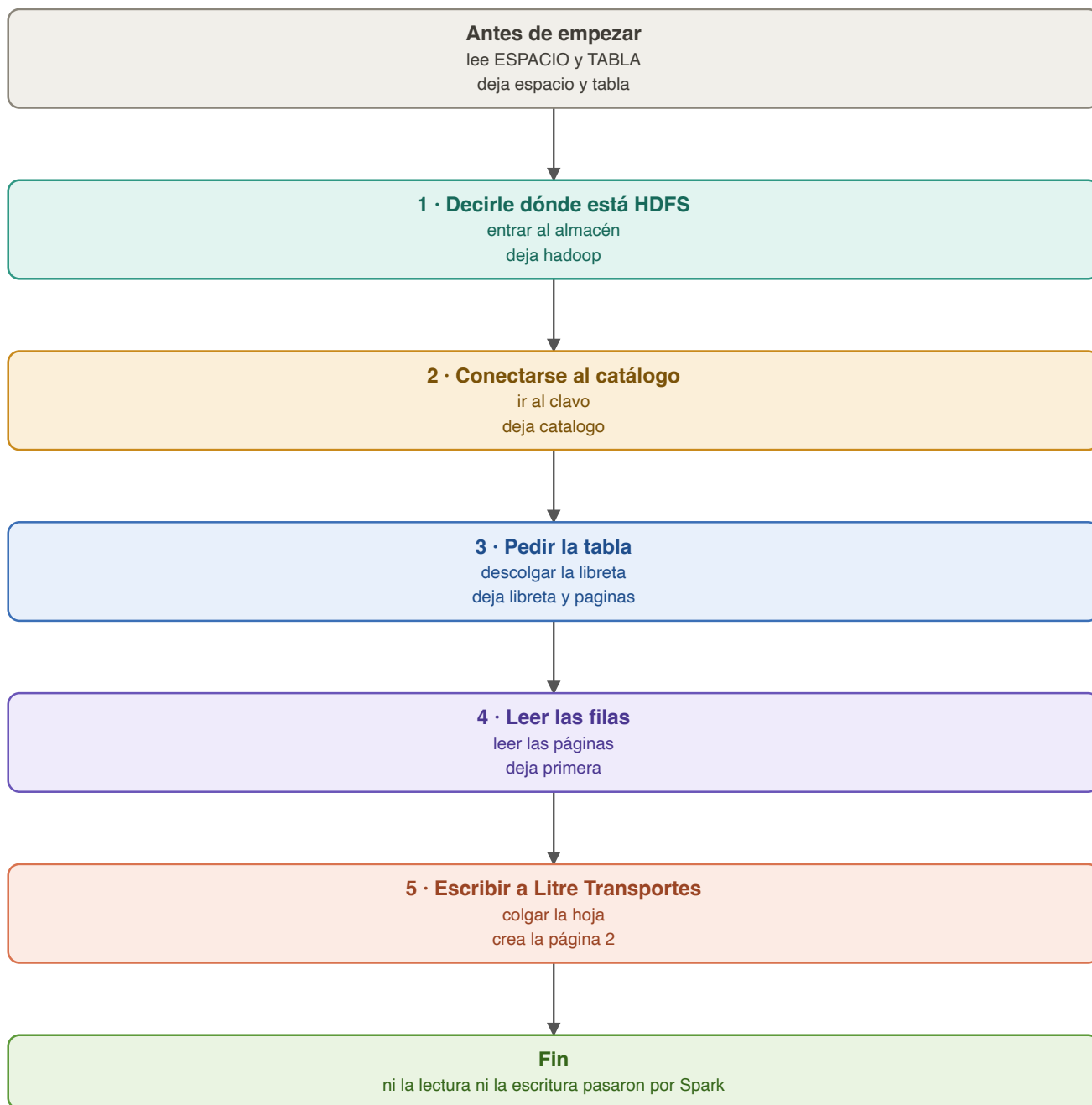
Paso 1 · El código, sin Spark

Este paso no tiene celdas que ejecutar. En el cuaderno está el programa `SinSpark.scala` completo, el mismo que corre el jar. Aquí va la idea general, y en las preguntas frecuentes está el código entero explicado línea por línea.

La pregunta. ¿Qué hace exactamente el programa que vas a correr?

Por qué ahora. Antes de correr un programa contra tu tabla, se lee. Son cinco movimientos, cada uno deja algo guardado para el siguiente, y en ninguno se habla con un servidor de Iceberg, porque ese servidor no existe.

En el almacén. Es el contador explicando lo que va a hacer antes de entrar a la bodega. Uno, entra al almacén. Dos, va al clavo. Tres, descuelga la libreta y mira la portada. Cuatro, lee las páginas. Cinco, escribe su hoja y la cuelga.



Antes de empezar (gris). El programa no trae escrito el nombre de ninguna tabla. Lo saca de dos variables de entorno, `ESPACIO` y `TABLA`, que le pone la celda que lo lanza. Los guarda en `espacio` y `tabla`.

1 · Decirle dónde está HDFS (verde agua). Arma una configuración de Hadoop con la dirección del almacenamiento y la guarda en `hadoop`.

2 · Conectarse al catálogo (amarillo). Crea un `HiveCatalog`, le pasa `hadoop` y le dice dónde está el Metastore. Queda guardado en `catalogo`.

3 · Pedir la tabla (azul). Le pide a `catalogo` la tabla. El resultado es `libreta`. De la libreta saca todas sus páginas, ordenadas por hora, en `paginas`. Todavía no leyó ni una fila, solo la portada.

4 · Leer las filas (morado). Lee la tabla como está ahora y después como estaba en la primera página, que guarda en `primera`. Es viajar en el tiempo sin motor de consultas.

5 · Escribir a Litre Transportes (coral). Si Litre no está, escribe un papelito Parquet nuevo con su fila y hace el commit. Eso crea la página 2.

Fin (verde). Cada movimiento dejó guardado algo para el siguiente, y ninguna importación del programa dice `org.apache.spark`.

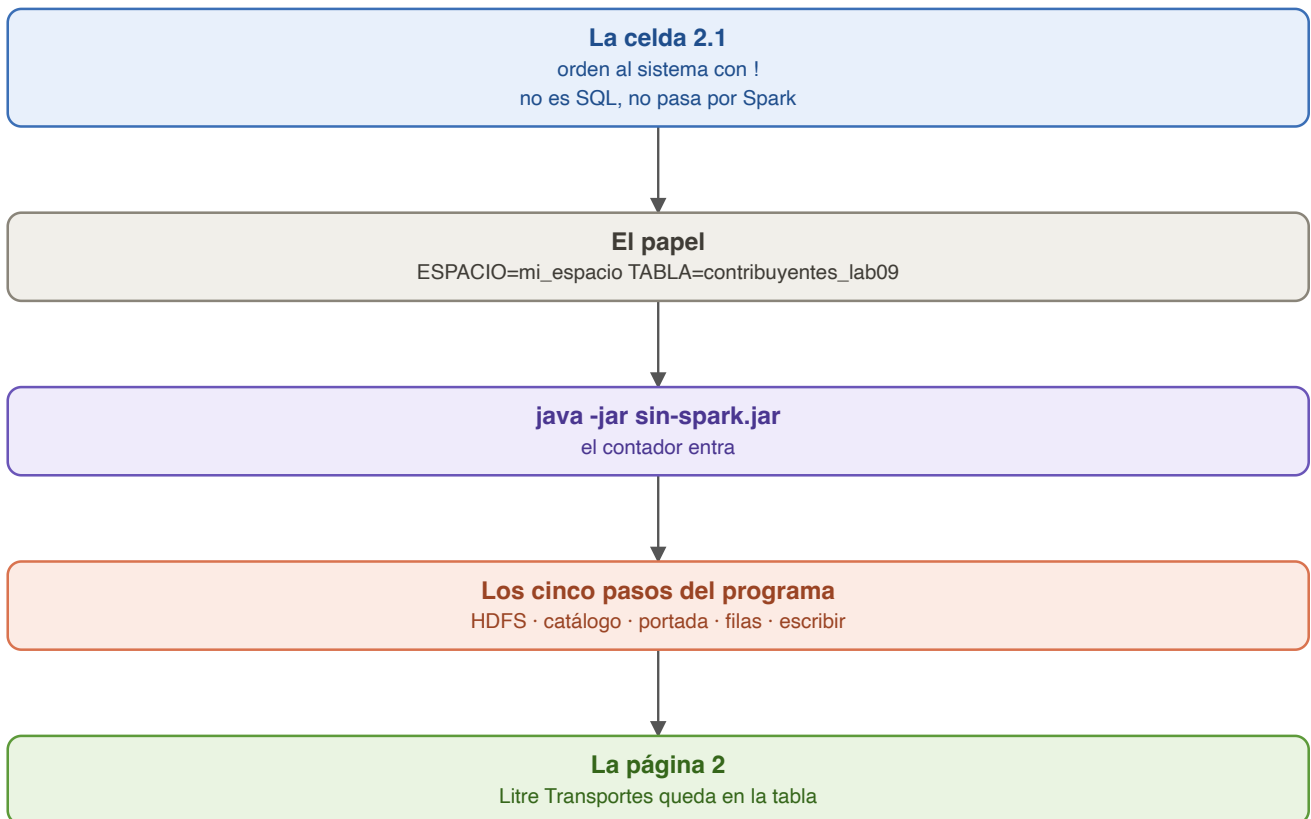
Paso 2 · Correrlo

Celda 2.1 · Correr el programa

La pregunta. ¿Un programa sin Spark puede leer y escribir tu tabla?

Por qué ahora. Ya sabes lo que hace el programa. Ahora lo corres contra tu tabla para ver si hace lo que dijo. La celda viene escrita en el cuaderno y no hay que cambiarle nada.

En el almacén. Es abrirle la puerta al contador y pasarle un papel que dice estante `mi_espacio`, libreta `contribuyentes_lab09`. Él entra solo, sin el almacenero, y va diciendo en voz alta cada cosa que hace.



La celda 2.1 (azul). No lleva `%%sql` porque no es una consulta. Empieza con `!`, que le dice al cuaderno que la línea no va a Spark sino directo al sistema operativo del contenedor. Por eso el comentario va con `#`, que es el comentario de Python, y no con dos guiones.

El papel (gris). `ESPACIO=mi_espacio` y `TABLA=contribuyentes_lab09` son variables de entorno. Escritas delante de un programa, existen solo para ese programa y solo mientras corre.

java -jar sin-spark.jar (morado). `java -jar` ejecuta un programa empaquetado en un jar, sin instalar nada. Un jar es un programa Java en un solo archivo, con todo lo que necesita adentro. `/opt/herramientas/sin-spark.jar` es donde el ambiente dejó el programa.

Los cinco pasos del programa (coral). Desde aquí corre lo que leíste en el paso 1, en el mismo orden, y cada paso imprime su número.

La página 2 (verde). Cuando aparece `Commit hecho`, Litre Transportes ya está en la tabla y la libreta tiene dos páginas.

La sentencia, parte por parte.

- `# Celda 2.1` es un comentario, que el cuaderno no ejecuta.
- `!` manda el resto de la línea al sistema operativo.
- `ESPACIO=mi_espacio` pone la variable de entorno `ESPACIO`, que el programa lee para saber en qué espacio está la tabla.
- `TABLA=contribuyentes_lab09` pone la variable `TABLA`, con el nombre de la tabla.
- `java -jar /opt/herramientas/sin-spark.jar` ejecuta el programa que está en esa ruta.

```
bash
# Celda 2.1
!ESPACIO=mi_espacio TABLA=contribuyentes_lab09 java -jar /opt/herramientas/sin-spark.jar
```

Lo que sale en pantalla.

```
text
Tabla pedida: mi_espacio.contribuyentes_lab09

1. HDFS en hdfs://namenode:8020
2. Catalogo Hive en thrift://hive-metastore:9083

3. Portada leída. Columnas: rut, razon_social, segmento
Paginas de la libreta, en hora UTC, igual que .snapshots:
2266374146400284423  2026-09-25 18:08:49.190  append

4. Las filas, leídas registro a registro y sin motor de consultas.
Pagina vigente:
76171162-8  Huemul Alimentos EIRL      GRANDE
77746521-K  Pehuen Logistica EIRL       MICRO
77884562-8  Araucaria Ferreteria SpA    PEQUENA
78800840-6  Copihue Maquinarias EIRL    MEDIANA
4 filas
Primera pagina, la 2266374146400284423:
76171162-8  Huemul Alimentos EIRL      GRANDE
77746521-K  Pehuen Logistica EIRL       MICRO
77884562-8  Araucaria Ferreteria SpA    PEQUENA
78800840-6  Copihue Maquinarias EIRL    MEDIANA
4 filas

5. Parquet escrito:
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab09/data/sin-spark-
a3f48485-af46-43f9-8742-3108d43fc10f.parquet
Lista de manifiestos de esa pagina:
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab09/metadata/snap-
2188129110206532514-1-9e598506-005e-4c96-a836-77d9a3d12317.avro
Commit hecho. Pagina nueva 2188129110206532514.

Fin. Ni la lectura ni la escritura pasaron por Spark. No hay Spark en este jar.
```

Cómo se lee. El programa encontró tu tabla con una página y cuatro filas, escribió a Litre Transportes y dejó una página nueva, la `2188129110206532514`. Todo sin Spark.

Ahora línea por línea. El programa escribe sin tildes a propósito, para que ningún sistema tenga problemas con los caracteres.

- Tabla pedida: `mi_espacio.contribuyentes_lab09` es lo que leyó de las variables de entorno.
- 1. HDFS en `hdfs://namenode:8020` dice dónde está el almacenamiento. `namenode` es el nombre del servidor de HDFS dentro del ambiente y `8020` su puerto.
- 2. Catalogo Hive en `thrift://hive-metastore:9083` dice dónde está el catálogo. `thrift` es el protocolo con que se habla con el Metastore, `hive-metastore` el nombre del servidor y `9083` su puerto.
- 3. Portada leída. Columnas: `rut`, `razon_social`, `segmento` dice que leyó la portada de la tabla y qué columnas tiene.
- Paginas de la libreta, en hora UTC, igual que `.snapshots` presenta la lista de páginas. Hay una sola, la `2266374146400284423`, de las 18:08:49.190 UTC, las 15:08 en Chile, con operación `append`, que significa que se agregaron filas. Es el `INSERT` de la celda 0.4.
- 4. Las filas, leídas registro a registro y sin motor de consultas presenta la lectura. `Pagina vigente` son las filas de hoy, y `Primera pagina`, la `2266374146400284423` las de esa página. Salen las mismas cuatro dos veces, con `4 filas` al pie, porque hasta ese momento la vigente y la primera son la misma página.
- 5. `Parquet escrito` da la ruta del papelito nuevo. Queda en la carpeta `data` de la tabla, y su nombre empieza con `sin-spark-` seguido de un identificador al azar.
- `Lista de manifiestos de esa pagina` da la ruta de la lista de manifiestos de la página nueva, en la carpeta `metadata`. Es el archivo `snap-2188129110206532514-1-...avro`, que dice qué manifiestos forman esa página, y cada manifiesto dice qué papelitos cuentan.
- `Commit hecho`. `Pagina nueva 2188129110206532514.` dice que colgó la página en el clavo.
- `Fin`. Ni la lectura ni la escritura pasaron por Spark. cierra el programa.

Las líneas en blanco entre pasos son las que imprime el propio programa con `println()`.

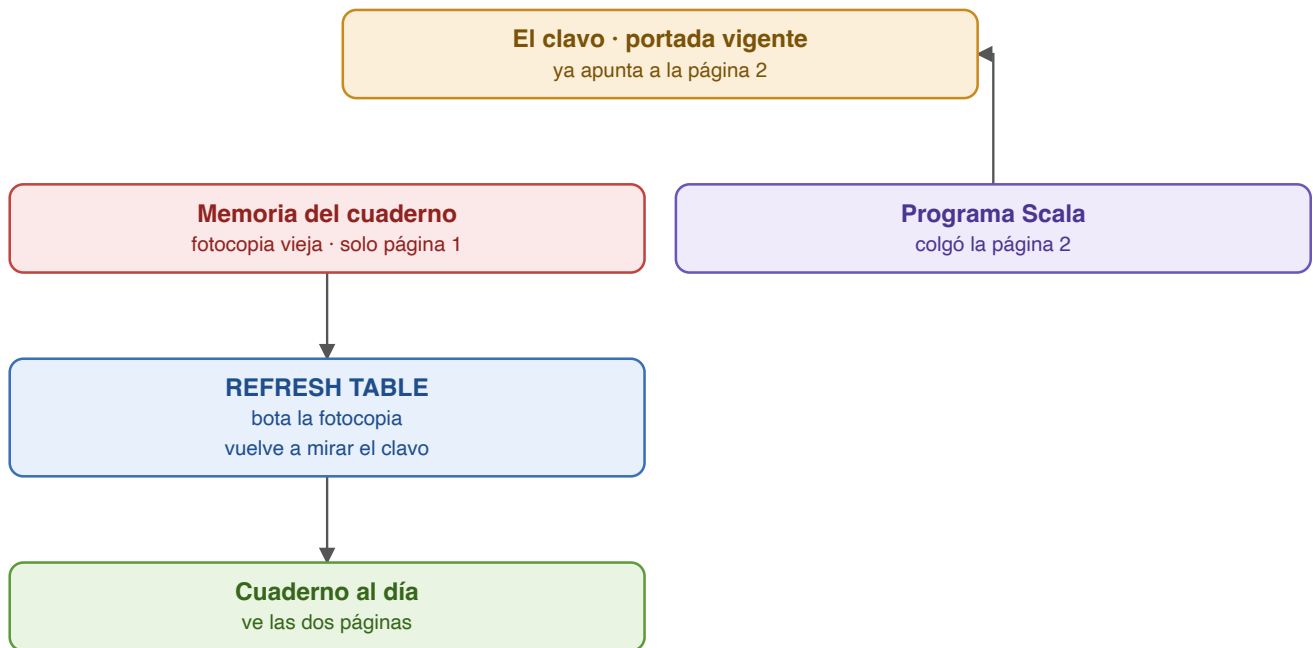
Paso 3 · Comprobar en SQL

Celda 3.1 · Refrescar la tabla

La pregunta. ¿Tu cuaderno ya sabe que el programa escribió?

Por qué ahora. La fila la escribió otro proceso. Tu cuaderno puede tener en memoria la portada de antes del programa, y si consultas de inmediato podrías ver cuatro filas y una página. No porque el dato se haya perdido, sino porque el cuaderno está atrasado.

En el almacén. El almacenero sacó una fotocopia de la portada hace un rato y la tiene en el mostrador para no ir al clavo cada vez. Mientras tanto, el contador colgó una hoja nueva. La fotocopia ya no sirve, y hay que ir al clavo a mirar la portada de verdad.



El clavo · portada vigente (amarillo). El catálogo ya está al día. Cuando el programa hizo el commit, el clavo quedó apuntando a la portada nueva.

Programa Scala (morado). Colgó la página 2 y terminó sin avisarle a nadie. En Iceberg nadie le avisa a nadie, cada uno mira el clavo.

Memoria del cuaderno (rojo). Para no ir al catálogo en cada consulta, Spark guarda en memoria la portada que leyó la última vez y la reutiliza por un rato. Esa es la fotocopia vieja, y en ella solo existe la página 1.

REFRESH TABLE (azul). Es esta celda. Le dice a Spark que bote la fotocopia y vuelva a mirar el clavo.

Cuaderno al día (verde). Después del refresco, el cuaderno ve lo mismo que dice el clavo.

La sentencia, parte por parte.

- `%%sql` vuelve porque esta celda sí es SQL.
- `REFRESH TABLE` le ordena a Spark que olvide lo que tenía guardado de la tabla y la lea de nuevo desde el catálogo.
- `contribuyentes_lab09` es la tabla, sin el espacio adelante porque sigues en `mi_espacio`.

```
sql
%%sql
-- Celda 3.1
REFRESH TABLE contribuyentes_lab09
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Listo. Desde aquí el cuaderno ve la tabla como la dejó el programa.

Celda 3.2 · Buscar la fila del programa

La pregunta. ¿La fila que escribió el programa Scala está en tu tabla?

Por qué ahora. El programa dijo que escribió, y no hay que creerle de palabra. Se comprueba con SQL. Si aparece Litre Transportes, la escribió un programa sin Spark y la está leyendo Spark.

En el almacén. El almacenero descuelga la libreta y la lee completa. Encuentra la hoja que dejó el contador.

La sentencia, parte por parte.

- `SELECT *` trae todas las columnas.
- `FROM contribuyentes_lab09` es la tabla de hoy.
- `ORDER BY rut` ordena por RUT, para que el resultado salga siempre igual.

```
sql
%%sql
-- Celda 3.2
SELECT * FROM contribuyentes_lab09 ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut          razon_social          segmento
76171162-8   Huemul Alimentos EIRL          GRANDE
77746521-K   Pehuen Logistica EIRL          MICRO
77884562-8   Araucaria Ferreteria SpA       PEQUENA
78800840-6   Copihue Maquinarias EIRL       MEDIANA
79856201-3   Litre Transportes SpA          MICRO
5 filas.
```

Cómo se lee. Cinco filas. Las cuatro de la celda 0.4 y la quinta, Litre Transportes SpA, RUT 79856201-3, segmento MICRO. Esa fila no la escribió tu cuaderno. Spark la lee igual que las otras, porque para la tabla no hay diferencia.

Celda 3.3 · Mirar las páginas

La pregunta. ¿Cuántas páginas tiene la libreta ahora?

Por qué ahora. Si el programa hizo un commit de verdad, tiene que haber una página más, con la misma hora que imprimió.

En el almacén. Es mirar la lista de páginas de la libreta. Tiene que estar la tuya y la del contador.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué se hizo.
- `FROM mi_espacio.contribuyentes_lab09.snapshots` lee la vista de sistema `.snapshots`, que es la lista de páginas. El nombre va con tres partes, el espacio, la tabla y la vista.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 3.3
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab09.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
2266374146400284423  2026-09-25 18:08:49.190  append
2188129110206532514  2026-09-25 18:08:52.227  append
2 filas.
```

Cómo se lee. Dos páginas, las dos `append`.

- La `2266374146400284423`, de las 18:08:49.190 UTC, es tu `INSERT`. Es la misma que el programa mostró en su paso 3.
- La `2188129110206532514`, de las 18:08:52.227 UTC, las 15:08 en Chile, es la del programa Scala. Es el número que imprimió en `Commit hecho`.

Para la libreta las dos páginas son iguales. No queda marca de qué programa escribió cada una.

Paso 4 · Las dependencias

Este paso no tiene celdas. En el cuaderno aparece lo que habría que declarar para hacer lo mismo en un proyecto Scala propio.

La pregunta. ¿Qué necesita un proyecto para leer y escribir tablas Iceberg sin Spark?

En el almacén. Es la lista de lo que el contador lleva en el maletín para poder entrar. La biblioteca que sabe leer la libreta, el cliente que sabe hablar con el clavo y el que sabe abrir la bodega.

```
scala
"org.apache.iceberg" % "iceberg-core"           % "1.3.0"
"org.apache.iceberg" % "iceberg-data"           % "1.3.0"
"org.apache.iceberg" % "iceberg-parquet"        % "1.3.0"
"org.apache.iceberg" % "iceberg-hive-metastore" % "1.3.0"
"org.apache.hive"     % "hive-metastore"        % "3.1.3"
"org.apache.hadoop"  % "hadoop-client"         % "3.3.6"
```

Cómo se lee. Son seis líneas, en el formato de sbt, la herramienta con que se arman los proyectos Scala. Cada línea dice quién publica la biblioteca, cómo se llama y qué versión se usa.

- `iceberg-core` es el núcleo de Iceberg, que sabe leer portadas, páginas y manifiestos.
- `iceberg-data` trae la lectura y escritura de registros, como `IcebergGenerics` y `GenericRecord`.
- `iceberg-parquet` sabe escribir y leer los papelitos en formato Parquet.
- `iceberg-hive-metastore` trae el `HiveCatalog`, que habla con el clavo.
- `hive-metastore` es el cliente del Metastore que el `HiveCatalog` usa por dentro.
- `hadoop-client` es el cliente de HDFS, porque los archivos viven ahí.

No hay un servidor que instalar ni un servicio que levantar.

Paso 5 · El mismo programa con Spark

Tampoco tiene celdas. El cuaderno muestra cómo se conecta un Spark cualquiera, por ejemplo el de un proceso nocturno, al mismo catálogo. No se ejecuta.

```
scala
val spark = SparkSession
  .builder()
  .appName("cliente-scala-con-spark")
  .master("local[*]")
  .config("spark.sql.extensions",
    "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
  .config("spark.sql.catalog.spark_catalog",
    "org.apache.iceberg.spark.SparkSessionCatalog")
  .config("spark.sql.catalog.spark_catalog.type", "hive")
  .config("spark.sql.catalog.spark_catalog.uri", "thrift://hive-metastore:9083")
  .config("spark.sql.catalogImplementation", "hive")
  .config("spark.sql.warehouse.dir", "hdfs://namenode:8020/warehouse/iceberg")
  .config("spark.hadoop.hive.metastore.uris", "thrift://hive-metastore:9083")
  .config("spark.hadoop.hive.metastore.warehouse.dir",
    "hdfs://namenode:8020/warehouse/iceberg")
  .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:8020")
  .config("spark.hadoop dfs.client.use.datanode.hostname", "true")
  .enableHiveSupport()
  .getOrCreate()

spark.sql("SELECT * FROM mi_espacio.contribuyentes_lab09 ORDER BY rut").show()
```

Cómo se lee. `SparkSession.builder()` empieza a armar una sesión de Spark. `appName` le pone nombre y `master("local[*]")` le dice que corra en la misma máquina, con todos sus núcleos. Las líneas `config` hacen tres cosas. Activan las extensiones de Iceberg en el SQL de Spark, con `spark.sql.extensions`. Le dicen que su catálogo es Iceberg sobre el Hive Metastore, con `spark_catalog`, su tipo `hive` y la dirección `thrift://hive-metastore:9083`. Y le dicen dónde está el almacenamiento, `hdfs://namenode:8020`. `enableHiveSupport()` usa el Metastore para las tablas, `getOrCreate()` crea la sesión y la última línea consulta la tabla y muestra el resultado con `show()`.

La diferencia entre leer Iceberg con Spark y sin Spark son esas líneas de configuración, no una arquitectura distinta.

Preguntas frecuentes

¿De dónde saca el programa espacio, tabla, libreta y primera?

Son nombres que el propio programa crea con `val`, que en Scala quiere decir guardo esto con este nombre y no lo cambio más.

- `espacio` y `tabla` salen de las variables de entorno `ESPACIO` y `TABLA`. `sys.env` es la lista de variables de entorno que recibió el programa, y `getOrElse` busca cada una por su nombre y, si no está, devuelve un texto vacío. Las variables las pone la celda 2.1 delante de `java -jar`.
- `libreta` sale de pedirle la tabla al catálogo con `catalogo.loadTable(TableIdentifier.of(espacio, tabla))`. Es el objeto que representa tu tabla, con su esquema, sus páginas y la ubicación de sus archivos.
- `primera` sale de `paginas.head.snapshotId()`. `paginas` es la lista de páginas ordenada por hora, `head` su primer elemento, la página más vieja, y `snapshotId()` su número. En la solución fue `2266374146400284423`.

¿Qué credenciales usa el programa?

Ninguna. En este ambiente HDFS está en modo simple, que confía en el usuario del sistema operativo con que corre el programa y no lo verifica. El Metastore tampoco pide clave. Por eso el programa solo necesita saber dónde están los servidores.

En una plataforma con Kerberos, como la de la DGT, eso no alcanza. El programa tendría que presentar una credencial antes de hablar con HDFS y con el Metastore. Haría falta lo siguiente.

- Un **principal**, que es el nombre de la cuenta de servicio en Kerberos, por ejemplo `cargas@DGT.CL`.
- Un **keytab**, que es un archivo con la clave de ese principal, para que el programa entre sin que nadie escriba una contraseña.
- La configuración de Kerberos de la máquina, el archivo `krb5.conf`, que dice dónde está el servidor de Kerberos.
- En la configuración de Hadoop, `hadoop.security.authentication` en `kerberos`, y un inicio de sesión con el keytab antes de todo lo demás, con `UserGroupInformation.loginUserFromKeytab`.
- En el catálogo, `hive.metastore.sasl.enabled` en `true` y el principal del Metastore en `hive.metastore.kerberos.principal`. SASL es la forma en que el cliente y el Metastore se autentican con Kerberos.

Con eso el programa tendría la llave. Qué tablas puede leer o escribir lo siguen decidiendo los permisos del catálogo y de HDFS, no Iceberg.

¿Qué es Litre Transportes?

Es un contribuyente de ejemplo, inventado como los otros cuatro. Litre Transportes SpA, RUT `79856201-3`, segmento `MICRO`. Es la fila que escribe el programa. Lleva nombre de árbol nativo, igual que los demás contribuyentes de la tabla, y en el resultado se reconoce de inmediato como la fila que no escribió el cuaderno.

¿Qué es la lista de manifiestos que imprime el programa?

Cada página de la libreta tiene una lista de manifiestos, un archivo `.avro` en la carpeta `metadata`. Esa lista dice qué manifiestos forman la página, y cada manifiesto dice qué papelitos cuentan en ella. Es la ruta que el programa imprime en su paso 5. El nombre del archivo empieza con `snap-` y el número de la página.

¿Qué pasa si corro la celda 2.1 dos veces?

El paso 5 revisa primero si Litre Transportes ya está en la tabla. Si está, imprime que el contribuyente ya estaba y no escribe nada. La celda se puede correr las veces que quieras sin duplicar la fila.

¿Por qué hace falta el REFRESH TABLE?

Porque Spark guarda en memoria la portada que leyó, para no ir al catálogo en cada consulta. La fila la escribió otro programa, y Spark no se entera hasta que vuelve a mirar el clavo. `REFRESH TABLE` lo obliga a mirar de inmediato. El dato estaba escrito desde el `Commit` hecho, el atrasado era el cuaderno.

¿Cómo sé que el programa no tiene Spark?

Porque ninguna de sus importaciones empieza con `org.apache.spark`. Todas empiezan con `java`, `scala`, `org.apache.hadoop`, `org.apache.iceberg` u `org.apache.parquet`. Y el jar se arma sin Spark entre sus dependencias.

¿Por qué el programa escribe sin tildes?

Para que la salida se lea bien en cualquier terminal y con cualquier configuración de caracteres. Es una decisión del programa, no una limitación de Iceberg.

El código completo de SinSpark.scala, línea por línea

Este es el archivo `cliente-scala/src/main/scala/cl/sii/iceberg/SinSpark.scala` del repositorio, completo y en orden, en tramos. Debajo de cada tramo va lo que hace cada línea.

```
scala
package cl.sii.iceberg
```

- `package cl.sii.iceberg` dice a qué paquete pertenece el programa. Un paquete es una carpeta de nombres, para que no choque con otro programa que se llame igual.

```
scala
import java.time.Instant
import java.time.ZoneOffset
import java.time.format.DateTimeFormatter
import java.util.UUID
import java.util.function.{Function => FuncionJava}

import scala.collection.JavaConverters._
```

- `import java.time.Instant` trae `Instant`, que representa un momento exacto en el tiempo.
- `import java.time.ZoneOffset` trae `ZoneOffset`, que representa un huso horario. Se usa para mostrar las horas en UTC.
- `import java.time.format.DateTimeFormatter` trae el formateador de fechas, que convierte un momento en texto.
- `import java.util.UUID` trae `UUID`, que genera identificadores al azar. Se usa para el nombre del papelito nuevo.
- `import java.util.function.{Function => FuncionJava}` trae la interfaz `Function` de Java y le cambia el nombre a `FuncionJava`, para no confundirla con las funciones de Scala.

- `import scala.collection.JavaConverters._` trae las conversiones entre las colecciones de Java y las de Scala. De ahí salen los `.asScala` y `.asJava` del programa.

```
scala
import org.apache.hadoop.conf.Configuration
import org.apache.iceberg.PartitionSpec
import org.apache.iceberg.catalog.TableIdentifier
import org.apache.iceberg.data.{GenericRecord, IcebergGenerics, Record}
import org.apache.iceberg.data.parquet.GenericParquetWriter
import org.apache.iceberg.expressions.Expressions
import org.apache.iceberg.hive.HiveCatalog
import org.apache.iceberg.io.CloseableIterable
import org.apache.iceberg.parquet.{Parquet, ParquetValueWriter}
import org.apache.parquet.schema.MessageType
```

- `Configuration` es la configuración de Hadoop, donde se anota dónde está HDFS.
- `PartitionSpec` describe cómo está particionada una tabla. Aquí se usa para decir que no lo está.
- `TableIdentifier` arma el nombre completo de una tabla, espacio y tabla.
- `GenericRecord`, `IcebergGenerics` y `Record` son la forma de Iceberg de leer y armar filas sin motor de consultas.
- `GenericParquetWriter` sabe convertir un registro en una fila de Parquet.
- `Expressions` arma filtros, como rut igual a tal valor.
- `HiveCatalog` es el catálogo de Iceberg que usa el Hive Metastore como clavo.
- `CloseableIterable` es una lista de filas que se lee de a una y que hay que cerrar al terminar.
- `Parquet` y `ParquetValueWriter` escriben papelitos Parquet.
- `MessageType` es el esquema de un archivo Parquet.

```
scala
/**
 * La misma tabla, sin Spark en ninguna parte.
 *
 * Este jar no lleva Spark adentro y esa ausencia es el punto: una tabla
 * Iceberg es un directorio de archivos Parquet mas unos manifiestos que dicen
 * cuales cuentan, y cualquier programa que sepa leer ese formato la lee. No
 * hace falta el motor que la escribio, ni ningun servidor que preste el dato.
 *
 * Se lee de arriba abajo, en cinco pasos numerados, y cada cosa se crea en el
 * paso anterior al que la usa. Es lo que el alumno tiene delante en el
 * laboratorio 09: esto es exactamente lo que corre el jar.
 */
```

- Todo lo que va entre `/**` y `*/` es un comentario largo. Dice que el jar no lleva Spark, que una tabla Iceberg es un directorio de archivos Parquet más unos manifiestos que dicen cuáles cuentan, y que el programa se lee de arriba abajo en cinco pasos.

```
scala
object SinSpark {

  def main(argumentos: Array[String]): Unit = {
```

- `object SinSpark` define el programa. En Scala un `object` es un objeto único, que existe desde que parte el programa.

- `def main(argumentos: Array[String]): Unit` es la función por donde empieza a correr. `argumentos` serían los argumentos de la línea de comandos, que aquí no se usan, y `Unit` quiere decir que no devuelve nada.

```
scala
// El espacio y la tabla los pone quien lanza el programa, nunca este
// archivo: así el mismo programa sirve para cualquier tabla, y ninguna
// queda amarrada a un nombre escrito aquí.
val espacio = sys.env.getOrElse("ESPACIO", "")
val tabla = sys.env.getOrElse("TABLA", "")
if (espacio.isEmpty || tabla.isEmpty) {
  println("Faltan las variables ESPACIO y TABLA. El programa se lanza así:")
  println("  ESPACIO=mi_espacio TABLA=contribuyentes_lab09 java -jar sin-spark.jar")
  return
}
println(s"Tabla pedida: $espacio.$tabla")
```

- Las tres líneas con `//` son un comentario. Dicen que el espacio y la tabla los pone quien lanza el programa.
- `val espacio = sys.env.getOrElse("ESPACIO", "")` lee la variable de entorno `ESPACIO`. Si no está, guarda un texto vacío.
- `val tabla = sys.env.getOrElse("TABLA", "")` hace lo mismo con `TABLA`.
- `if (espacio.isEmpty || tabla.isEmpty)` pregunta si falta alguna de las dos. `||` quiere decir o.
- Los dos `println` de adentro explican cómo se lanza el programa, y `return` lo termina sin hacer nada más.
- `println(s"Tabla pedida: $espacio.$tabla")` imprime la primera línea de la salida. La `s` antes de las comillas permite meter variables con `$`.

```
scala
// 1. Decirle donde esta HDFS. La direccion es la del ambiente.
val hadoop = new Configuration()
hadoop.set("fs.defaultFS", "hdfs://namenode:8020")
hadoop.set("dfs.client.use.datanode.hostname", "true")
hadoop.set("hive.metastore.uris", "thrift://hive-metastore:9083")
println()
println("1. HDFS en hdfs://namenode:8020")
```

- `val hadoop = new Configuration()` crea una configuración de Hadoop vacía.
- `hadoop.set("fs.defaultFS", "hdfs://namenode:8020")` anota la dirección de HDFS.
- `hadoop.set("dfs.client.use.datanode.hostname", "true")` le dice al cliente que llegue a los servidores de datos por su nombre y no por su dirección interna, que es lo que funciona dentro de este ambiente.
- `hadoop.set("hive.metastore.uris", "thrift://hive-metastore:9083")` anota dónde está el Metastore.
- `println()` imprime una línea en blanco, y la línea siguiente imprime el paso 1 de la salida.

```
scala
// 2. Conectarse al catalogo, que es el clavo del que cuelga la libreta.
val catalogo = new HiveCatalog()
catalogo.setConf(hadoop)
catalogo.initialize("laboratorio", Map(
  "uri" -> "thrift://hive-metastore:9083",
  "warehouse" -> "hdfs://namenode:8020/warehouse/iceberg").asJava)
println("2. Catalogo Hive en thrift://hive-metastore:9083")
```

- `val catalogo = new HiveCatalog()` crea el catálogo.
- `catalogo.setConf(hadoop)` le pasa la configuración de Hadoop.
- `catalogo.initialize("laboratorio", Map(...).asJava)` lo deja listo. "laboratorio" es el nombre que se le da, `uri` la dirección del Metastore y `warehouse` la carpeta base de las tablas en HDFS. `.asJava` convierte el `Map` de Scala en uno de Java, que es lo que pide Iceberg.
- El `println` imprime el paso 2 de la salida.

scala

```
// 3. Pedir la tabla. Esto lee la portada y todavia ningun dato.
val libreta = catalogo.loadTable(TableIdentifier.of(espacio, tabla))
val reloj = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS").withZone(ZoneOffset.UTC)
val paginas = libreta.snapshots().asScala.toSeq.sortBy(_.timestampMillis())
println()
println("3. Portada leida. Columnas: " +
  libreta.schema().columns().asScala.map(_.name()).mkString(", "))
println("  Paginas de la libreta, en hora UTC, igual que .snapshots:")
paginas.foreach(pagina => println("    %s %s %s".format(
  pagina.snapshotId(),
  reloj.format(Instant.ofEpochMilli(pagina.timestampMillis())),
  pagina.operation())))
```

- `val libreta = catalogo.loadTable(TableIdentifier.of(espacio, tabla))` pide la tabla al catálogo. Lee la portada, todavía ningún dato.
- `val reloj = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS").withZone(ZoneOffset.UTC)` prepara el formato de las horas, año, mes, día, hora, minutos, segundos y milésimas, en UTC.
- `val paginas = libreta.snapshots().asScala.toSeq.sortBy(_.timestampMillis())` toma todas las páginas, las convierte en una lista de Scala y las ordena por hora. `_` es cada página.
- Las líneas de `println` imprimen el paso 3. `libreta.schema().columns()` son las columnas, `map(_.name())` saca sus nombres y `mkString(", ")` los junta con comas.
- `paginas.foreach(...)` recorre las páginas e imprime una línea por cada una, con su número, su hora formateada con `reloj` y su operación. `"%s"` es donde va cada valor.

scala

```
// 4. Leer las filas de la pagina vigente y de la primera pagina.
def mostrar(titulo: String, registros: CloseableIterable[Record]): Unit = {
  println("  " + titulo)
  try {
    val filas = registros.asScala.toSeq.map(registro => "    %-12s %-26s %s".format(
      registro.getField("rut"), registro.getField("razon_social"),
      registro.getField("segmento")))
    filas.sorted.foreach(println)
    println("    %d filas".format(filas.size))
  } finally registros.close()
}
```

- `def mostrar(titulo: String, registros: CloseableIterable[Record]): Unit` define una función auxiliar que imprime un título y unas filas.
- `println(" " + titulo)` imprime el título con tres espacios adelante.
- `try { ... } finally registros.close()` asegura que las filas se cierren al terminar, pase lo que pase.
- `registros.asScala.toSeq.map(...)` convierte cada registro en una línea de texto. `getField` saca el valor de cada columna, y `"%-12s %-26s %s"` los alinea a la izquierda en anchos de 12 y 26 caracteres.

- `filas.sorted.foreach(println)` ordena las líneas y las imprime.
- `println(" %d filas".format(filas.size))` imprime cuántas filas eran. `%d` es donde va el número.

```
scala
println()
println("4. Las filas, leídas registro a registro y sin motor de consultas.")
mostrar("Pagina vigente:", IcebergGenerics.read(libreta).build())
if (paginas.nonEmpty) {
  val primera = paginas.head.snapshotId()
  mostrar(s"Primera pagina, la $primera:",
    IcebergGenerics.read(libreta).useSnapshot(primera).build())
}
```

- Las dos primeras líneas imprimen una línea en blanco y el encabezado del paso 4.
- `mostrar("Pagina vigente:", IcebergGenerics.read(libreta).build())` lee la tabla como está ahora e imprime sus filas.
- `if (paginas.nonEmpty)` solo sigue si la tabla tiene alguna página.
- `val primera = paginas.head.snapshotId()` guarda el número de la página más vieja.
- `IcebergGenerics.read(libreta).useSnapshot(primera).build()` lee la tabla como estaba en esa página, y `mostrar` la imprime.

```
scala
// 5. Escribir la fila de Litre Transportes: el Parquet, el manifiesto y el commit.
val rut = "79856201-3"
val buscado = IcebergGenerics.read(libreta).where(Expressions.equal("rut", rut)).build()
val yaEsta = try buscado.iterator().hasNext finally buscado.close()
println()
if (yaEsta) {
  println(s"5. El contribuyente $rut ya estaba en la tabla. No se escribe de nuevo.")
  println(" Esta celda se puede correr las veces que quieras.")
}
```

- `val rut = "79856201-3"` es el RUT de Litre Transportes.
- `val buscado = IcebergGenerics.read(libreta).where(Expressions.equal("rut", rut)).build()` busca en la tabla las filas con ese RUT.
- `val yaEsta = try buscado.iterator().hasNext finally buscado.close()` pregunta si encontró alguna, y cierra la búsqueda.
- Si ya estaba, los dos `println` lo dicen y el programa no escribe nada.

```
scala
} else {
  val registro = GenericRecord.create(libreta.schema())
  registro.setField("rut", rut)
  registro.setField("razon_social", "Litre Transportes SpA")
  registro.setField("segmento", "MICRO")
}
```

- `else` es el camino cuando Litre no está.
- `val registro = GenericRecord.create(libreta.schema())` crea una fila vacía con las columnas de la tabla.
- Las tres líneas `registro.setField` llenan el RUT, la razón social y el segmento.

```
scala
val destino = libreta.io().newOutputFile(
  libreta.locationProvider().newDataLocation(s"sin-spark-${UUID.randomUUID()}.parquet"))
val escritor = Parquet.writeData(destino)
  .schema(libreta.schema())
  .createWriterFunc(new FuncionJava[MessageType, ParquetValueWriter[_]] {
    override def apply(tipo: MessageType): ParquetValueWriter[_] =
      GenericParquetWriter.buildWriter(tipo)
  })
  .withSpec(PartitionSpec.unpartitioned())
  .overwrite()
  .build[Record]()
try escritor.write(registro) finally escritor.close()
```

- `val destino = libreta.io().newOutputFile(libreta.locationProvider().newDataLocation(...))` elige dónde va el papelito nuevo, dentro de la carpeta `data` de la tabla, con un nombre que empieza con `sin-spark-` y sigue con un `UUID` al azar.
- `val escritor = Parquet.writeData(destino)` empieza a armar el escritor del papelito.
- `.schema(libreta.schema())` le da las columnas de la tabla.
- `.createWriterFunc(new FuncionJava[...] {...})` le dice cómo convertir cada registro en Parquet, usando `GenericParquetWriter.buildWriter`.
- `.withSpec(PartitionSpec.unpartitioned())` dice que la tabla no está particionada.
- `.overwrite()` permite escribir aunque ya exista un archivo con ese nombre, que en la práctica no pasa por el `UUID`.
- `.build[Record]()` termina de armar el escritor.
- `try escritor.write(registro) finally escritor.close()` escribe la fila y cierra el papelito.

```
scala
println(s"5. Parquet escrito: ${destino.location()}")
libreta.newAppend().appendFile(escritor.toDataFile).commit()
libreta.refresh()
println(s"  Lista de manifiestos de esa pagina:
${libreta.currentSnapshot().manifestListLocation()}")
println(s"  Commit hecho. Pagina nueva ${libreta.currentSnapshot().snapshotId()}")
}
```

- El primer `println` imprime la ruta del papelito escrito.
- `libreta.newAppend().appendFile(escritor.toDataFile).commit()` es el commit. `newAppend` prepara una página que agrega archivos, `appendFile` le suma el papelito nuevo y `commit` la cuelga en el clavo.
- `libreta.refresh()` vuelve a leer la portada, para ver la página recién colgada.
- Los dos `println` imprimen la ruta de la lista de manifiestos de la página nueva y su número.

```
scala
println()
println("Fin. Ni la lectura ni la escritura pasaron por Spark. No hay Spark en este jar.")
}
}
```

- `println()` y la línea que sigue imprimen el cierre de la salida.
- Las dos llaves de abajo cierran la función `main` y el objeto `SinSpark`.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

La tabla no le pertenece a ningún motor. Un programa que no es Spark la abrió, la leyó, le escribió una fila, y Spark la leyó después sin enterarse de nada distinto.

En el almacén. El contador entró con su llave, miró el clavo, descolgó la libreta, leyó y anotó su hoja con las mismas reglas que el almacenero. Nadie le prestó nada y nadie tuvo que exportarle nada.

Los números de la solución ejecutada.

Momento	Filas	Páginas	Quién escribió
Después de la celda 0.4	4	1	el cuaderno, con Spark
Después de la celda 2.1	5	2	el programa Scala, sin Spark

Cualquier programa que cargue la biblioteca de Iceberg, tenga la llave y sepa dónde está el clavo lee y escribe la misma tabla. Eso permite que una plataforma cambie de motor sin mover los datos, y que una herramienta nueva se conecte sin que nadie le abra una puerta especial.